GraphQL Schema, Query, Mutation, and Core Concepts: Visual & Detailed Guide

---

1. GraphQL Schema: The Blueprint

A **schema** defines the types, queries, mutations, and relationships in your API. It is the contract between frontend and backend.

Visual Representation:

```
+-------------------+
|   GraphQL Schema  |
+-------------------+
|  type User        |
|  type Product     |
|  type Query       |
|  type Mutation    |
+-------------------+
```

Example:

```graphql
type User {
  id: ID!
  name: String!
  email: String!
}

type Product {
  id: ID!
  name: String!
  price: Float!
}

type Query {
  users: [User!]!
  products: [Product!]!
}

type Mutation {
  addUser(name: String!, email: String!): User!
  addProduct(name: String!, price: Float!): Product!
}
```

---

2. Query: Fetching Data

A **query** is how the client asks for data. You specify exactly what you want.

Visual Example:

```
Client ----(query)---> GraphQL Server ----> DB
         <---(data)---
```

Example:

```
query GetUsers {
  users {
    id
    name
    email
  }
}
```

Result:

```
{
  "data": {
    "users": [
      { "id": "1", "name": "Alice", "email": "alice@example.com" },
      { "id": "2", "name": "Bob", "email": "bob@example.com" }
    ]
  }
}
```

---

3. Mutation: Modifying Data

A **mutation** is how the client changes data (create, update, delete).

Visual Example:

```
Client --(mutation)--> GraphQL Server --(write)--> DB
         <---(result)---
```

Example:

```
mutation AddUser($name: String!, $email: String!) {
  addUser(name: $name, email: $email) {
    id
    name
    email
  }
}
```

Variables:

```
{
  "name": "Charlie",
```

```
  "email": "charlie@example.com"
}
```

Result:

```
{
  "data": {
    "addUser": {
      "id": "3",
      "name": "Charlie",
      "email": "charlie@example.com"
    }
  }
}
```

---

4. Subscription: Real-time Data

A **subscription** lets the client get real-time updates (e.g., new orders).

Visual Example:

```
Client <===(WebSocket)===> GraphQL Server
```

Example:

```
subscription OnUserAdded {
  userAdded {
    id
    name
    email
  }
}
```

---

5. Resolvers: The Logic Layer

Resolvers are functions that fetch or modify data for each field in the schema.

Visual Example:

```
Query: users
   |
   v
Resolver: users() { ...fetch from DB... }
```

---

6. Introspection: Self-Documenting

GraphQL APIs can be introspected to discover types, queries, and mutations.

Visual Example:

```
Client --(introspection query)--> GraphQL Server
         <---(schema info)---
```

---

7. End-to-End Example: E-Commerce

Schema:

- User, Product, Order types
- Query: users, products, orders
- Mutation: addUser, addProduct, placeOrder

Query:

```
query GetOrders {
  orders {
    id
    user { name }
    products { name price }
    total
  }
}
```

Mutation:

```
mutation PlaceOrder($userId: ID!, $productIds: [ID!]!) {
  placeOrder(userId: $userId, productIds: $productIds) {
    id
    total
  }
}
```

See the companion HLD/LLD doc for a full e-commerce architecture with diagrams.